# Fruit Ninja Robot: A Comparison of Motion Planning and Reinforcement Learning for Dynamic Object Interception

**Stephanie Mei, Megan Tseng, Michelle Wang**

## 1 Motivation

Projectile interception is a challenging and applicable dynamic manipulation problem. The robot must compute and move to an appropriate pose all on the order of milliseconds, making planning, control, and trajectory optimization important objectives. We choose a simulated 3D fruit ninja task as a testbed to investigate the robotic projectile interception problem.

Executing the 3D fruit ninja task involves planning how to intercept fruits with a bladed end effector, control the blade arm to fully slice through the fruit, and move the end effector between consecutive fruits at different locations. Different approaches like reinforcement learning (RL) and trajectory optimization have been used to coordinate these objectives, and they present different tradeoffs in performance, computation cost, and engineering effort. This project aims to implement and compare these methods using the fruit ninja task.

## 2 Prior work

There are two primary approaches in the literature that tackle the projectile interception problem: motion planning and reinforcement learning.

One approach is motion planning for projectile interception, which involves finding a collision-free path to move from one position to the projectile position in a specific timeframe. [1] uses sampling-based motion planning (SBMP) to perform the fruit ninja task in the real world. Since slicing is different from other catching tasks, the planner considers the end effector's velocity and orientation, as well as dependencies between intercept point arrival time and fruit trajectory. We will implement a heavily simplified version of this system in simulation. This work uses tossed objects, while we will be using vertically dropped fruits. In addition, this planner is deployed on the Franka Research 3, and the paper details how hardware limits on torque, power, and joint configuration are factored in. These considerations of hardware limits will be applicable to adapting the slicing problem to our chosen platform.

Another approach is RL. RL learns directly from data and maximizing rewards, as opposed to sampling c-space to determine a path. Notably, RL has been able to solve tasks involving dexterous manipulation. [2] uses sim2real reinforcement learning to enable a robot arm to catch and stabilize a ball on a flat paddle. Fruit ninja is a similar task of interception at a certain end-effector pose but does not involve grasping. We will implement a simplified and modified version of this system for fruit slicing.

Past work also explores using vision-based approaches to collect information about the robot's environment. [3] leverages RGB cameras, convolutional neural networks, and machine learning techniques to predict the trajectory and arrival time of a rolling ball. Due to the scope of the project and limited hardware access, we will not be using a vision-based approach and assume a privileged state.

## 3 Proposed approach

Concretely, the minimal product is to plan and execute a path to intercept a projectile with an observable trajectory. To do so, we will compare motion planning and RL approaches. The key comparisons between the two approaches and primary research questions will be:

1. Performance: **which approach has a higher success rate in slicing fruits?**
2. Cost: **How much computation is required for both approaches?** Computation will be captured by software profiling tools within our codebase. **How much time does each approach take?** Time is measured as cumulative time needed to perform the task end-to-end. We will break time measurements down further into (1) time for processing/computation and (2) time for motion traversal.
3. Suitability to problem: **How much engineering effort was required for each approach?** To measure this, we will keep track of time spent to formulate each approach, qualitatively explain obstacles we encounter for each approach, note whether further assumptions had to be made, and evaluate overall how feasible the approach was despite any obstacles or assumptions.

### 3.1 Environment

First, we will set up the fruit ninja environment in MuJuCo/MJLab with the **kuka iiwa** assets. We will model the fruit as a spherical rigid body and define an interception/cutting condition based on the contact force and location between the fruit and the robot's end effector. In this setup, the fruit will free-fall from a known reachable location. Since gravity is the only force at play, we can predict the fruit's trajectory.

### 3.2 Algorithms

#### 3.2.1 Trajectory Optimization

We will use direct collocation (Hermite-Simpson) where we formulate interception of the fruit as a free-final-time trajectory optimization problem in joint space. Collocation will enforce dynamics at each node, joint limits, and the chosen cutting condition for the blade reaching the fruit's position. Because this final position depends on arrival time, we will wrap the solver in an outer search over candidate arrival times, solving a fixed-time collocation problem for each. Initialization for each solve will be provided by an **IK** solution for a blade pose that meets the fruit's predicted pose, and interpolating from cht eucrrent configuration to seed the trajectory.

#### 3.2.2 Reinforcement Learning

For RL, we will define a sparse task reward for intercepting the fruit and meeting the cutting condition. We may also need to introduce auxiliary reward terms like action rate, etc. to regularize motion quality. Both motion planning and **RL** will output end-effector poses and use **IK** to generate motor commands.

### 3.3 Project Scope Considerations

This project is scoped exclusively for simulation where we can make the environment as perfect or as noisy as needed. We also plan to use privileged state for RL policy/motion planner input, simplifying state estimation and eliminating the need for vision systems. We plan to approach this project incrementally. RL for manipulation can be very finicky to tune. To build intuition and solve a simple case first, we can intercept objects with only vertical downward velocity before handling objects with 3D trajectories. Motion planning can be computationally slow. To test our system before optimizing, we can slow down time in simulation. We expect to use the MIT Engaging GPU cluster for RL training.

## 4 Evaluation and Expected Outcomes

### 4.1 Evaluation

For evaluation metrics, we will define success as being able to intercept the projectile at a certain end-effector velocity and angle. Since our simulation will use privileged information about the projectile, we can check the states of the end-effector and projectile at the intercept point. We will use a binary metric based on whether it intercepts the midpoint of the projectile within a specified tolerance.

### 4.2 Expected Outcomes

We aim to make a practical comparison of methods from an ease of use and implementation perspective, providing *a practitioner's guide* on the costs and benefits of choosing trajectory optimization or reinforcement learning on a simplified 2D slicing problem. Aside from binary success of intercepting fruits, important outcomes include time needed to compute solutions via each method and robustness to noise.

### 4.3 Extensions

If time allows, we will explore how our project can be expanded to meet different stretch goals, including dropping a higher volume of fruits, throwing fruits with 3D velocity, and using a variety of fruit types. It would also be interesting to explore different problem setups and discover if motion planning is good at some of them while RL excels in others.

## 5 Projected Timeline

Table 1: Project timeline and distribution of labor

| Date | Milestone | Owner(s) |
|---|---|---|
| Oct 26 | Finish proposal. | All |
| Nov 2 | Understand dynamics of fruit chopping, propose chopping criteria, familiarize with MuJoCo, and set up simulator in shared GitHub repository. | All |
| Nov 9 | Implementation for trajectory optimization and RL (does not need to work yet). | TrajOpt: Megan<br>RL: Michelle, Stephanie |
| Nov 16 | Working proof of concept; determine feasibility of RL. | TrajOpt: Megan<br>RL: Michelle, Stephanie |
| Nov 20 | Design evaluation experiments for the paper. Have draft of paper ready for CI peer review. | All (each writes on their own methods and experiments) |
| Nov 23 | If successful, fine-tune and expand scope. If not, use as debugging buffer time. | All |
| Nov 30 | Film project video and finalize report. | All |

Since the simulation infrastructure is shared, all members will contribute code throughout the project.

## References

[1] L. Chen, A. Garrett, A. Niu, and Z. Kingston. Becoming a fruit ninja: Real-time probabilistic kinodynamic planning for manipulator projectile interception, 2026. URL https://arxiv.org/abs/2609.22608.

[2] K. Ren, G. Wang, A. S. Morgan, and K. Hang. Zero-shot sim-to-real robot learning: A dexterous manipulation study on reactive catching, 2026. URL https://arxiv.org/abs/2605.09789.

[3] N. Lee, Y. Oh, and J. Moon. Catching robot: Predicting the trajectory of a rolling ball using transformer. *IEEE Access*, 12:128551–128558, 01 2024. doi:10.1109/ACCESS.2024.3455553.